In [ ]:
```python
# Imports partilhados: ficam visíveis em todas as células e nas
# funções/classes decoradas (e, por isso, importáveis pelos testes).
import random
import base64
import tempfile
from pathlib import Path


import marimo as mo
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter
import numpy as np
import z3
from contextlib import redirect_stdout
from io import StringIO
```

1. Importação das bibliotecas

Inicialmente, são importadas as bibliotecas necessárias ao desenvolvimento e execução do projeto:

- **random:** permite gerar valores aleatórios, nomeadamente para inicializar as células da borda e escolher uma célula a atualizar no modo assíncrono.
- **marimo:** permite construir e executar a aplicação interativa, através de células e botões.
- **matplotlib.pyplot:** utilizada para criar e apresentar representações gráficas das grelhas.
- **FuncAnimation e PillowWriter:** permitem criar animações da evolução do Game of Life e guardá-las em formato GIF.
- **numpy:** permite converter as grelhas em matrizes, facilitando a sua representação gráfica.
- **z3:** biblioteca utilizada para a modelação simbólica do problema e verificação das propriedades através de um solver SMT.
- **base64, tempfile e Path:** utilizadas para guardar temporariamente as animações, ler os ficheiros GIF gerados e codificá-los de forma a serem apresentados na aplicação Marimo.

2. Implementação do motor do Game of Life

O motor é implementado através da classe `GameOfLife_Engine`, responsável por representar e executar a evolução do autómato celular.

O construtor recebe dois parâmetros: `N`, que define a dimensão da grelha normal, e `modo`, que determina o tipo de atualização utilizado. São considerados dois modos:

- **SYNC (síncrono):** todas as células são atualizadas simultaneamente, tendo em conta o estado da grelha anterior.
- **ASYNC (assíncrono):** apenas uma célula é atualizada em cada transição.

2.1. Classificação das células

O método `tipo(i, j)` identifica o tipo de cada posição da grelha:

- **Inicial:** células pertencentes à borda inicial.
- **Normal:** células que constituem a grelha principal, sujeitas às regras do Game of Life.
- **Final:** células pertencentes à borda final.

A grelha tem dimensão `(N+2) × (N+2)`, incluindo assim a borda inicial, a grelha normal e a borda final.

2.2. Inicialização do estado

O método `estado_inicial(borda=None, vivas=None)` constrói o estado inicial da grelha.

A borda inicial pode ser gerada aleatoriamente, definida por um valor constante (0 ou 1) ou através de um dicionário com valores específicos para cada posição.

Relativamente às células normais, é possível utilizar a configuração inicial definida no código, correspondente a um bloco 3×3 de células vivas no canto superior esquerdo, ou indicar explicitamente quais as células que começam vivas através do parâmetro `vivas`.

As células da borda final são inicializadas com o valor zero.

2.3. Contagem dos vizinhos

O método `conta_vizinhos(g, i, j)` calcula o número de células vivas na vizinhança de uma determinada célula.

São consideradas as oito posições adjacentes (incluindo diagonais), excluindo a própria célula.

2.4. Regra de evolução

O método `proximo_valor(g, i, j)` implementa as regras do Conway's Game of Life (B3/S23):

- Uma célula morta com exatamente três vizinhos vivos nasce.
- Uma célula viva com dois ou três vizinhos vivos sobrevive.

- Nos restantes casos, a célula morre ou permanece morta.

2.5. Geração dos sucessores

O método `sucessores(g)` calcula os possíveis estados seguintes da grelha, dependendo do modo de execução.

No modo síncrono (`SYNC`), todas as células normais são atualizadas simultaneamente, utilizando sempre os valores do estado anterior. É devolvida uma lista com uma única grelha sucessora.

No modo assíncrono (`ASYNC`), são construídas todas as possibilidades de atualização: cada sucessor corresponde à atualização de uma única célula normal, mantendo todas as restantes inalteradas. Assim, são devolvidos `N × N` sucessores possíveis.

2.6. Execução de um passo

O método `passo(g)` executa uma única transição do motor.

No modo síncrono, utiliza o único sucessor produzido pelo método `sucessores()

2.7. Contagem de células vivas e deteção de erro

O método c_vivas(grid) contabiliza o número de células vivas exclusivamente na grelha normal, ignorando as bordas.

O método estado_erro(grid) verifica se a grelha se encontra num estado de erro, isto é, se não existe nenhuma célula normal viva.

In [ ]:
```
# Parâmetros do trabalho
N = 18
MODO = "ASYNC"   # "SYNC" (todas em simultâneo) ou "ASYNC" (uma de cada vez)
```

In [ ]:
```
class GameOfLife_Engine:
    def __init__(self, N, modo):
        assert N > 16
        assert modo in ("SYNC", "ASYNC")
        self.N = N
        self.modo = modo

    def tipo(self, i, j):
        N = self.N
        if max(i, j) == N + 1:    # um índice é N+1 -> final
            return "final"
        if min(i, j) == 0:        # um índice é 0 (e nenhum é N+1) -> inicial
            return "inicial"
        return "normal"
```

```
    def estado_inicial(self, borda=None, vivas=None):
        """
        borda: None -> aleatória | int (0/1) -> constante | dict {(i,j): valor}
        vivas: None -> regra do enunciado (bloco 3x3)
               iterável de (i,j) -> só essas casas normais começam vivas
        """
        N = self.N

        if vivas is not None:
            vivas = set(vivas)
            for (i, j) in vivas:
                if not (1 <= i <= N and 1 <= j <= N):
                    raise ValueError(f"({i},{j}) não é uma casa normal")

        grelha = [[0] * (N + 2) for _ in range(N + 2)]
        for i in range(N + 2):
            for j in range(N + 2):
                t = self.tipo(i, j)
                if t == "inicial":
                    if borda is None:
                        grelha[i][j] = random.randint(0, 1)
                    elif isinstance(borda, dict):
                        grelha[i][j] = borda[(i, j)]
                    else:
                        grelha[i][j] = borda          # valor constante
                elif t == "normal":
                    if vivas is None:
                        grelha[i][j] = 1 if (i <= 3 and j <= 3) else 0
                    else:
                        grelha[i][j] = 1 if (i, j) in vivas else 0
        return grelha

    def conta_vizinhos(self, g, i, j):
        vivos = 0
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                if (dx, dy) != (0, 0):
                    vivos += g[i + dx][j + dy]
        return vivos

    def proximo_valor(self, g, i, j):
        v = self.conta_vizinhos(g, i, j)
        if v == 3:
            return 1              # nasce ou sobrevive
        if v == 2:
            return g[i][j]        # mantém o valor atual
        return 0                  # morre ou continua morta

    def sucessores(self, g):
        N = self.N

        if self.modo == "SYNC":
            nova = [linha[:] for linha in g]              # mantém iniciais e finais
            for i in range(1, N + 1):
```

```
                for j in range(1, N + 1):
                    nova[i][j] = self.proximo_valor(g, i, j)   # lê sempre
de g (antiga)
            return [nova]

        elif self.modo == "ASYNC":
            resultado = []
            for i in range(1, N + 1):
                for j in range(1, N + 1):
                    nova = [linha[:] for linha in g]       # uma cópia por
escolha
                    nova[i][j] = self.proximo_valor(g, i, j)
                    resultado.append(nova)
            return resultado

        else:
            raise ValueError("modo inválido: usa 'SYNC' ou 'ASYNC'")

    def passo(self, g):
        """Um passo de simulação: devolve UMA grelha."""
        if self.modo == "SYNC":
            return self.sucessores(g)[0]
        # ASYNC: atualiza só uma célula aleatória
        i = random.randint(1, self.N)
        j = random.randint(1, self.N)
        nova = [linha[:] for linha in g]
        nova[i][j] = self.proximo_valor(g, i, j)
        return nova

    def c_vivas(self, grid):
        N = len(grid) - 2
        return sum(sum(linha[1:N + 1]) for linha in grid[1:N + 1])

    def estado_erro(self, grid):
        return self.c_vivas(grid) == 0
```

3. Animações do Game of Life

Para visualizar a evolução das figuras no motor, foi criada a função `animacao_motor()`.

Esta função recebe o motor, o número de passos da simulação, a configuração das células vivas e o valor da borda.

Inicialmente, é gerado o estado inicial da grelha. De seguida, o motor executa sucessivos passos, guardando cada estado para construir a animação.

A evolução é representada graficamente através da biblioteca Matplotlib, utilizando `FuncAnimation`. Em cada frame, a grelha é atualizada e o título apresenta o número do passo, a quantidade de células vivas e o modo de execução.

Por fim, a animação é guardada temporariamente em formato GIF e apresentada diretamente na aplicação Marimo através de `mo.Html`.

3.1. Botões de execução das figuras

Foram criados botões individuais no Marimo para permitir executar o motor com diferentes configurações iniciais:

- Glider;
- Blinker;
- Relógio;
- T-tetromino;
- Serpente;
- Cruz.

Cada botão está associado a uma célula independente, que executa a função `animacao_motor()` com a respetiva configuração. Desta forma, é possível observar individualmente a evolução de cada figura e comparar o seu comportamento no motor.

Existe ainda um botão **Run Motor**, que permite executar o motor com a configuração inicial padrão.

In [ ]:
```python
# LLM inicio
def animacao_motor(engine,passos=30,vivas = None, borda=None):
    """Executa o motor e devolve uma animação GIF da evolução da figura."""
    estados = [engine.estado_inicial(borda=borda, vivas=vivas)]
    for _ in range(passos):
        estados.append(engine.passo(estados[-1]))

    fig, ax = plt.subplots(figsize=(5, 5))
    imagem = ax.imshow(
        np.asarray(estados[0]),
        cmap="gray",
        vmin=0,
        vmax=1,
        interpolation="nearest",
    )
    ax.axis("off")
    titulo = ax.set_title(
        f"Passo 0 | células vivas: {engine.c_vivas(estados[0])} | modo:
{engine.modo}"
    )

    def atualizar(passo):
        estado = estados[passo]
        imagem.set_data(np.asarray(estado))
        titulo.set_text(
            f"Passo {passo} | células vivas: {engine.c_vivas(estado)} |
modo: {engine.modo}"
        )
```

```
        return imagem, titulo

    animacao = FuncAnimation(
        fig,
        atualizar,
        frames=len(estados),
        interval=350,
        blit=True,
        repeat=True,
    )

    # O GIF é apresentado diretamente no Marimo e não depende de
JavaScript.
    with tempfile.TemporaryDirectory() as pasta_temporaria:
        caminho_gif = Path(pasta_temporaria) / "game_of_life.gif"
        animacao.save(caminho_gif, writer=PillowWriter(fps=3))
        gif_bytes = caminho_gif.read_bytes()
    plt.close(fig)
    gif = base64.b64encode(gif_bytes).decode("ascii")
    return mo.Html(
        f'<img src="{gif}" '
        f'alt="Animação do Game of Life" '
        f'style="display:block;max-width:100%;height:auto;margin:auto;">'
    )
```

In [ ]:

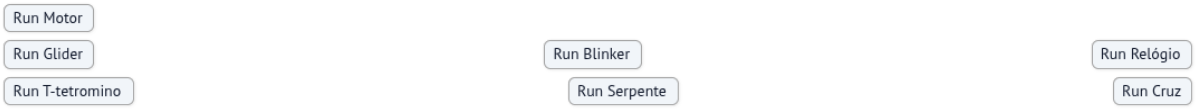
```
# Figuras disponíveis para executar individualmente no motor.
glider = [(2, 3), (3, 4), (4, 2), (4, 3), (4, 4)]
blinker = [(4, 4), (4, 5), (4, 6)]
relogio = [(8, 8), (9, 9), (9, 10), (10, 7), (10, 8), (11, 9)]   #
oscilador, período 2
t_tetromino = [(8, 8), (8, 9), (8, 10), (9, 9)]                  # ao passo
9 vira um semáforo, período 2 (12 células)
serpente = [(8, 7), (8, 8), (8, 10), (9, 7), (9, 9), (9, 10)]    # estática
(período 1)
cruz = [(8, 9), (9, 8), (9, 9), (9, 10), (10, 9)]                # ao passo
6 estabiliza num oscilador de período 2 (12 células)
```

In [ ]:

```
# Cada botão inicia apenas a simulação da figura correspondente.
run_motor = mo.ui.run_button(label="Run Motor")
run_glider = mo.ui.run_button(label="Run Glider")
run_blinker = mo.ui.run_button(label="Run Blinker")
run_relogio = mo.ui.run_button(label="Run Relógio")
run_t_tetromino = mo.ui.run_button(label="Run T-tetromino")
run_serpente = mo.ui.run_button(label="Run Serpente")
run_cruz = mo.ui.run_button(label="Run Cruz")

mo.vstack([
    mo.md("### Executar figuras no motor Game of Life"),
    mo.hstack([run_motor]),
    mo.hstack([run_glider, run_blinker, run_relogio]),
    mo.hstack([run_t_tetromino, run_serpente, run_cruz]),
])
```

Executar figuras no motor Game of Life



In [ ]:
```
mo.stop(not run_motor.value)
motor = GameOfLife_Engine(N, MODO)
animacao_motor(motor, passos=30)
```

In [ ]:
```
mo.stop(not run_glider.value)
motor_glider = GameOfLife_Engine(N, MODO)
animacao_motor(motor_glider,30,glider, borda=0)
```

In [ ]:
```
mo.stop(not run_blinker.value)
motor_blinker = GameOfLife_Engine(N, MODO)
animacao_motor(motor_blinker, 20,blinker, borda=0)
```

In [ ]:
```
mo.stop(not run_relogio.value)
motor_relogio = GameOfLife_Engine(N, MODO)
animacao_motor(motor_relogio, 20,relogio, borda=0)
```

In [ ]:
```
mo.stop(not run_t_tetromino.value)
motor_t_tetromino = GameOfLife_Engine(N, MODO)
animacao_motor(motor_t_tetromino, 30, t_tetromino, borda=0)
```

In [ ]:
```
mo.stop(not run_serpente.value)
motor_serpente = GameOfLife_Engine(N, MODO)
animacao_motor(motor_serpente,20, serpente,borda=0)
```

In [ ]:
```
mo.stop(not run_cruz.value)
motor_cruz = GameOfLife_Engine(N, MODO)
animacao_motor(motor_cruz, 30,cruz, borda=0)
# LLM fim
```

Construção da grelha simbólica — `grelha_z3()`

A função `grelha_z3(N, t)` constrói uma grelha simbólica em Z3, correspondente ao instante `t`.

Cada célula é representada por uma variável booleana, que pode assumir os valores verdadeiro ou falso, correspondentes a uma célula viva ou morta.

As células da borda inicial são representadas por variáveis livres (`Cons_i_j`), enquanto as células da borda final são definidas como falsas. As células normais recebem nomes que identificam a sua posição e o instante de tempo.

Esta função permite representar simbolicamente o estado da grelha em diferentes instantes da execução.

In [ ]:
```python
def grelha_z3(N, t):
    g_z3 = [[0] * (N + 2) for _ in range(N + 2)]

    for i in range(len(g_z3)):
        for j in range(len(g_z3)):
            if max(i, j) == N + 1:
                g_z3[i][j] = z3.BoolVal(False)
            elif min(i, j) == 0:
                g_z3[i][j] = z3.Bool(f"Cons_{i}_{j}")
            else:
                g_z3[i][j] = z3.Bool(f"n_{i}_{j}_{t}")

    return g_z3
```

Identificação dos vizinhos — `c_vizinhos()`

A função `c_vizinhos(g, i, j)` recolhe as oito células vizinhas de uma determinada posição da grelha simbólica.

São consideradas todas as posições adjacentes, incluindo as diagonais, excluindo a própria célula.

O resultado é uma lista de variáveis booleanas do Z3, que será utilizada na aplicação da regra de evolução do Game of Life.

In [ ]:
```python
def c_vizinhos(g, i, j):
    vizinhos = []

    for dx in (-1, 0, 1):
        for dy in (-1, 0, 1):
            if (dx, dy) != (0, 0):
                vizinhos.append(g[i + dx][j + dy])

    return vizinhos
```

Regra de evolução simbólica — `proximo_z3()`

A função `proximo_z3(x, vizinhos)` implementa simbolicamente a regra B3/S23 do Game of Life, utilizando restrições booleanas do Z3.

É utilizada a função `PbEq()` para verificar se existem exatamente três ou exatamente duas células vizinhas vivas.

Assim, uma célula fica viva se tiver exatamente três vizinhos vivos, ou se já estiver viva e tiver exatamente dois vizinhos vivos.

A função devolve uma expressão lógica que representa o valor da célula no instante seguinte.

In [ ]:
```python
def proximo_z3(x, vizinhos):
    """Regra B3/S23 com restrições pseudo-booleanas."""
    exatamente_3 = z3.PbEq([(v, 1) for v in vizinhos], 3)
    exatamente_2 = z3.PbEq([(v, 1) for v in vizinhos], 2)
    return z3.Or(exatamente_3, z3.And(x, exatamente_2))
```

Aplicação da regra a uma célula — `regra_z3()`

A função `regra_z3(g, i, j)` aplica a regra de evolução a uma célula específica da grelha simbólica.

Para isso, obtém o estado atual da célula e a respetiva vizinhança através das funções `c_vizinhos()` e `proximo_z3()`.

O resultado é uma expressão booleana que determina simbolicamente se a célula estará viva no instante seguinte.

In [ ]:
```python
def regra_z3(g, i, j):
    return proximo_z3(g[i][j], c_vizinhos(g, i, j))
```

Transição síncrona — `T_sync()`

A função `T_sync(N, t)` define a transição entre dois estados consecutivos da grelha no modo síncrono.

Inicialmente, são construídas duas grelhas simbólicas, correspondentes aos instantes `t` e `t+1`.

De seguida, para cada célula normal, é criada uma restrição que obriga o seu valor no instante seguinte a corresponder ao resultado da aplicação da regra do Game of Life ao estado atual.

Desta forma, todas as células são atualizadas simultaneamente, utilizando os valores do estado anterior.

In [ ]:
```python
def T_sync(N, t):
    g = grelha_z3(N, t)
    gp = grelha_z3(N, t + 1)
    restricoes = []

    for i in range(1, N + 1):
        for j in range(1, N + 1):
            prox = regra_z3(g, i, j)
            restricoes.append(gp[i][j] == prox)

    return z3.And(restricoes)
```

Transição assíncrona — `T_async()`

A função `T_async(N, t)` define a transição entre dois estados consecutivos no modo assíncrono.

É utilizada uma variável inteira (`sel`) para identificar a célula que será atualizada, garantindo que apenas uma célula normal é escolhida em cada transição.

Para cada célula, é criada uma restrição que determina o seu valor no instante seguinte: se for a célula selecionada, aplica-se a regra do Game of Life; caso contrário, mantém o valor atual.

Assim, o Z3 representa todas as possibilidades de atualização de uma única célula por transição.

In [ ]:
```python
def T_async(N, t):
    g = grelha_z3(N, t)
    gp = grelha_z3(N, t + 1)
    restricoes = []

    # Um único inteiro identifica a célula selecionada (0 .. N*N-1).
    # Evita criar N*N variáveis booleanas sel e uma restrição one-hot.
    sel = z3.Int(f"selidx_{t}")
    restricoes.extend([sel >= 0, sel < N * N])

    for i in range(1, N + 1):
        for j in range(1, N + 1):
            indice = (i - 1) * N + (j - 1)
            escolhida = (sel == indice)
            prox = regra_z3(g, i, j)

            # A selecionada atualiza; todas as outras mantêm o valor.
            restricoes.append(
                gp[i][j] == z3.If(escolhida, prox, g[i][j])
            )

    return z3.And(restricoes)
```

Seleção do tipo de transição — `T_main()`

A função `T_main(N, t, MODO)` permite selecionar a transição a utilizar, de acordo com o modo de execução definido.

Se o modo for `SYNC` , chama a função `T_sync()` . Se for `ASYNC` , chama a função `T_async()` .

Desta forma, esta função centraliza a escolha do comportamento do motor simbólico, permitindo utilizar a mesma estrutura de verificação para os dois modos.

In [ ]:
```python
def T_main(N, t, MODO):
    if MODO == "SYNC":
        return T_sync(N, t)
    elif MODO == "ASYNC":
        return T_async(N, t)
    else:
        raise ValueError("Modo inválido")
```

Estado inicial simbólico — `I_z3()`

A função `I_z3(N)` define as condições que a grelha deve satisfazer no instante inicial.

É criada uma grelha simbólica através de `grelha_z3()` e são adicionadas restrições para que as células normais formem um bloco 3×3 de células vivas no canto superior esquerdo, ficando as restantes células normais mortas.

As células da borda inicial permanecem livres, podendo assumir qualquer um dos valores booleanos.

O resultado é uma expressão lógica que representa todos os estados iniciais possíveis.

In [ ]:
```python
def I_z3(N):
    """Estado inicial: bloco 3x3 vivo, resto das normais morto; iniciais livres (aleatórias)."""
    g = grelha_z3(N, 0)
    return z3.And([g[i][j] == z3.BoolVal(i <= 3 and j <= 3)
                   for i in range(1, N + 1) for j in range(1, N + 1)])
```

Definição do estado de erro — `erro_z3()`

A função `erro_z3(N, t)` verifica simbolicamente se, num determinado instante, todas as células normais estão mortas.

Para isso, constrói a grelha correspondente ao instante `t` e utiliza uma expressão lógica que nega a existência de qualquer célula normal viva.

Assim, a função devolve uma condição que é verdadeira quando se verifica o estado de erro.

In [ ]:
```python
def erro_z3(N, t):
    g = grelha_z3(N, t)
    return z3.Not(z3.Or([g[i][j] for i in range(1, N + 1) for j in range(1, N + 1)]))
```

Extração de um traço — `extrair()`

A função `extrair(m, N, k)` permite obter os estados da grelha a partir de um modelo encontrado pelo solver Z3.

Percorre todos os instantes, desde 0 até `k`, avaliando o valor de cada célula normal e de cada célula da borda.

Os valores booleanos são convertidos em inteiros (0 ou 1), permitindo construir uma lista com a evolução completa da grelha até ao instante pretendido.

In [ ]:
```python
def extrair(m, N, k):
    tr = []
    for t in range(k + 1):
        g = grelha_z3(N, t)
        tr.append([[int(z3.is_true(m.eval(g[i][j], model_completion=True)))
                    for j in range(N + 2)] for i in range(N + 2)])
    return tr
```

Validação de um traço — `validatraco_z3()`

A função `validatraco_z3(engine, tr)` permite confirmar se um traço encontrado pelo Z3 é válido quando executado no motor concreto.

Primeiro, verifica se o estado inicial do traço corresponde às condições de inicialização do motor, incluindo os valores da borda inicial.

De seguida, percorre os estados consecutivos e confirma que cada estado seguinte pertence ao conjunto de sucessores possíveis do estado anterior.

Desta forma, garante-se que o contraexemplo encontrado pelo Z3 respeita as regras de funcionamento do motor concreto.

In [ ]:
```python
def validatraco_z3(engine, tr):
    """Re-executa o contra-exemplo no motor concreto."""
    N = engine.N
    assert tr[0] == engine.estado_inicial(borda={(i, j): tr[0][i][j]
                for i in range(N + 2) for j in range(N + 2) if engine.tipo(i,
j) == "inicial"})
    for t in range(len(tr) - 1):
        assert tr[t + 1] in engine.sucessores(tr[t]), f"passo {t} inválido"
    return True
```

4. Verificação da Propriedade 1 — Nunca se alcança o estado de erro

A primeira propriedade pretende verificar se o sistema consegue alguma vez atingir um estado de erro, ou seja, um estado em que todas as células normais estão mortas.

Para verificar esta propriedade, foi utilizada uma estratégia baseada em duas abordagens complementares:

1. Verificação de um candidato a invariante

Inicialmente, foi definido o candidato a invariante segundo o qual existe sempre pelo menos uma célula normal viva.

Para verificar se este candidato constitui efetivamente um invariante, são testadas duas condições:

- **Base:** verificar se todos os estados iniciais satisfazem o candidato a invariante.
- **Preservação:** verificar se qualquer transição possível, a partir de um estado que satisfaz o candidato, continua a satisfazê-lo.

Estas condições são verificadas através do solver Z3, procurando estados que violem cada uma delas.

2. Pesquisa de contraexemplos através de Bounded Model Checking (BMC)

Como a falha da condição de preservação do candidato a invariante não significa necessariamente que exista um estado de erro alcançável a partir das condições iniciais, foi utilizada uma segunda abordagem.

Através de BMC, procura-se encontrar um percurso que parta de um estado inicial válido e alcance um estado de erro, até um número máximo de transições definido por `kmax`.

O solver verifica sucessivamente se existe um estado de erro em cada instante, acrescentando as relações de transição necessárias para representar a evolução do sistema.

Caso seja encontrado um contraexemplo, o respetivo percurso é extraído e validado através do motor concreto.

Caso não seja encontrado nenhum contraexemplo até ao limite definido, o resultado é considerado inconclusivo, pois não existe uma prova de que o estado de erro nunca possa ser alcançado em qualquer instante.

Esta estratégia é aplicada ao modo de execução selecionado (`SYNC` ou `ASYNC`), permitindo analisar ambas as interpretações das transições do sistema.

Definição do candidato a invariante — `inv_p1()`

A função `inv_p1(g, N)` define o candidato a invariante utilizado na verificação da primeira propriedade.

O candidato corresponde à existência de pelo menos uma célula normal viva na grelha.

Para isso, é construída uma disjunção lógica (`Or`) de todas as células normais. Assim, a expressão é verdadeira sempre que pelo menos uma dessas células estiver viva.

Este candidato é utilizado posteriormente para verificar as condições de base e de preservação do invariante.

In [ ]:
```python
def inv_p1(g, N):
    """Candidato: existe pelo menos uma célula normal viva."""
    return z3.Or([
        g[i][j]
        for i in range(1, N + 1)
        for j in range(1, N + 1)
    ])
```

Verificação do candidato a invariante — `verifica_invariante_p1()`

A função `verifica_invariante_p1(N, MODO)` verifica se o candidato definido em `inv_p1()` satisfaz as condições necessárias para ser considerado um invariante.

São realizados dois testes independentes através do Z3:

- **Base:** procura um estado que satisfaça a condição inicial, mas que não satisfaça o candidato a invariante. Se o resultado for `UNSAT`, significa que todos os estados iniciais satisfazem o candidato.
- **Preservação:** procura uma transição em que o estado atual satisfaça o candidato, mas o estado seguinte não o satisfaça. Se o resultado for `UNSAT`, significa que o candidato é preservado por todas as transições possíveis.

No final, a função devolve os resultados dos dois testes e indica se o candidato constitui um invariante válido.

É importante notar que o teste de preservação considera todos os estados que satisfazem o candidato, não apenas os estados alcançáveis a partir da condição inicial.

In [ ]:
```python
def verifica_invariante_p1(N, MODO):
    """Verifica base e preservação do candidato Inv.

    Base:       I(s) AND NOT Inv(s) é UNSAT.
    Preservação: Inv(s) AND T(s,s') AND NOT Inv(s') é UNSAT.

    O passo é testado sobre todos os estados que satisfazem Inv,
    não apenas sobre estados já alcançáveis.
    """
    g0 = grelha_z3(N, 0)
    g1 = grelha_z3(N, 1)
```

```
    # 1. Base: todos os estados iniciais satisfazem Inv?
    solver_base = z3.Solver()
    solver_base.add(I_z3(N))
    solver_base.add(z3.Not(inv_p1(g0, N)))
    resultado_base = solver_base.check()

    # 2. Passo: qualquer sucessor de um estado Inv também satisfaz Inv?
    solver_passo = z3.Solver()
    solver_passo.add(inv_p1(g0, N))
    solver_passo.add(T_main(N, 0, MODO))
    solver_passo.add(z3.Not(inv_p1(g1, N)))
    resultado_passo = solver_passo.check()

    return {
        "base": resultado_base,
        "passo": resultado_passo,
        "base_ok": resultado_base == z3.unsat,
        "passo_ok": resultado_passo == z3.unsat,
        "invariante_valido": (
            resultado_base == z3.unsat
            and resultado_passo == z3.unsat
        ),
    }
```

Pesquisa de contraexemplos — `bmc_erro_p1()`

A função `bmc_erro_p1(N, MODO, kmax)` implementa a pesquisa de estados de erro através de Bounded Model Checking.

Inicialmente, adiciona ao solver a condição inicial do sistema. De seguida, percorre os instantes desde 0 até `kmax`, verificando se existe um estado de erro em cada instante.

Em cada iteração, o solver verifica a condição de erro para o instante atual. Se encontrar uma solução (`SAT`), extrai o modelo correspondente e devolve o percurso que conduz ao estado de erro.

Se não existir solução (`UNSAT`), acrescenta a relação de transição para avançar para o instante seguinte.

Caso o solver devolva `UNKNOWN`, a pesquisa é interrompida e é devolvido o motivo da indeterminação.

Se não for encontrado nenhum estado de erro até ao limite definido, a função devolve `UNSAT_LIMIT`, indicando que não foram encontrados contraexemplos dentro desse limite.

In [ ]:
```
def bmc_erro_p1(N, MODO, kmax):
    """Procura um estado de erro em 0..kmax transições.

    Devolve:
```

```
      ("SAT", k, traco)           se encontrou contraexemplo;
      ("UNSAT_LIMIT", None, None) se não encontrou até kmax;
      ("UNKNOWN", k, motivo)      se o Z3 não conseguiu decidir.
    """
    solver = z3.Solver()
    solver.add(I_z3(N))

    for k in range(kmax + 1):
        solver.push()
        solver.add(erro_z3(N, k))
        resultado = solver.check()

        if resultado == z3.sat:
            modelo = solver.model()
            traco = extrair(modelo, N, k)
            solver.pop()
            return "SAT", k, traco

        if resultado == z3.unknown:
            motivo = solver.reason_unknown()
            solver.pop()
            return "UNKNOWN", k, motivo

        solver.pop()

        if k < kmax:
            solver.add(T_main(N, k, MODO))

    return "UNSAT_LIMIT", None, None
```

Execução da verificação da Propriedade 1 — `p1_main()`

A função `p1_main(N, modo, kmax)` coordena as duas abordagens utilizadas na verificação da primeira propriedade.

Primeiro, executa `verifica_invariante_p1()` para testar o candidato a invariante, apresentando os resultados das condições de base e de preservação.

De seguida, executa `bmc_erro_p1()` para procurar um contraexemplo a partir dos estados iniciais.

Se for encontrado um contraexemplo, o percurso é extraído e validado através da função `validatraco_z3()`, utilizando o motor concreto. São também contabilizadas as células normais vivas em cada instante.

Se não for encontrado nenhum contraexemplo até ao limite estabelecido, a conclusão é apresentada como inconclusiva.

Por fim, os resultados dos testes são organizados num dicionário e devolvidos pela função.

In [ ]:

```
def p1_main(N,modo,kmax=9):
    """Executa a P1: invariante e procura de contraexemplo."""
    resultados = {}

    print(f"PROPRIEDADE 1 — MODO {modo}")


    # Vertente A: teste do candidato a invariante.
    inv = verifica_invariante_p1(N, modo)
    print("\n[1] Verificação do candidato Inv = existe uma célula viva")
    print("Base:", inv["base"])
    print("Passo indutivo:", inv["passo"])

    if inv["base_ok"]:
        print("  Base válida: todos os estados iniciais satisfazem Inv.")
    else:
        print("  Base inválida: existe um estado inicial que viola Inv.")

    if inv["passo_ok"]:
        print("  Preservação válida: todos os sucessores preservam Inv.")
    elif inv["passo"] == z3.sat:
        print("  Preservação falhou: existe uma transição Inv -> NOT Inv.")
        print("  Nota: esse estado pode não ser alcançável desde I.")
    else:
        print("  Preservação inconclusiva: o Z3 devolveu UNKNOWN.")

    print(
        "  Candidato indutivo:",
        "SIM" if inv["invariante_valido"] else "NÃO"
    )

    # Vertente B: BMC a partir dos estados iniciais do enunciado.
    print(f"\n[2] BMC: procura de estado de erro até ao passo {kmax}")
    status, k, detalhe = bmc_erro_p1(N, modo, kmax)

    traco = None
    if status == "SAT":
        traco = detalhe
        motor = GameOfLife_Engine(N, modo)
        valido = validatraco_z3(motor, traco)
        contagens = [motor.c_vivas(g) for g in traco]

        print("  CONTRAEXEMPLO ENCONTRADO.")
        print("  Primeiro passo de erro encontrado:", k)
        print("  Células normais vivas por instante:", contagens)
        print("  Traço validado no motor concreto:", valido)
        print("  Conclusão: a propriedade 1 é FALSA neste modo.")

    elif status == "UNSAT_LIMIT":
        print(f"  Nenhum erro encontrado até {kmax} transições.")
        print("  Conclusão: INCONCLUSIVO (BMC limitado não é uma prova
global).")

    else:
        print("  O Z3 devolveu UNKNOWN no passo:", k)
```

```
        print("  Motivo:", detalhe)
        print("  Conclusão: INCONCLUSIVO.")

    resultados[modo] = {
        "invariante": inv,
        "bmc_status": status,
        "passo_erro": k,
        "traco": traco,
    }

    return resultados
```

5. Verificação da Propriedade 2 — Nenhuma célula normal está permanentemente morta ou permanentemente viva

A segunda propriedade pretende verificar se cada célula normal consegue assumir ambos os estados possíveis ao longo da evolução do sistema: viva e morta.

Para verificar esta propriedade, foi utilizada uma estratégia baseada em duas abordagens complementares: simulação do motor concreto e pesquisa simbólica através do Z3.

1. Pesquisa de testemunhos através de simulação

Inicialmente, são realizadas várias simulações do motor concreto, utilizando diferentes configurações da borda inicial.

Durante estas simulações, é registado, para cada célula normal, se já foi observada viva e se já foi observada morta.

São utilizadas 200 configurações da borda: duas constantes (1 e 0) e 198 configurações geradas aleatoriamente.

No modo síncrono, cada simulação é executada durante 150 passos. No modo assíncrono, são realizados 40 varrimentos da grelha, atualizando as células uma de cada vez.

Desta forma, procura-se obter testemunhos que demonstrem que cada célula consegue assumir ambos os valores.

2. Pesquisa simbólica através do Z3

Após as simulações, verifica-se quais as células para as quais ainda não foi encontrado um testemunho vivo ou morto.

Para essas células, é utilizada a função `alcanca_viva()`, que recorre ao solver Z3 para procurar um percurso a partir das condições iniciais em que a célula assume o

estado pretendido.

A pesquisa é limitada por um número máximo de transições (`kmax`). Sempre que é encontrado um percurso, este é extraído e validado através do motor concreto.

No final, a propriedade é considerada verificada na interpretação existencial utilizada pelo código quando todas as células normais possuem testemunhos de ambos os estados.

3. Verificação de um estado de erro absorvente

Adicionalmente, é realizada uma pesquisa de um estado de erro absorvente, isto é, um estado em que todas as células normais estão mortas e permanecem mortas independentemente da transição escolhida.

Esta pesquisa permite analisar uma interpretação mais forte da propriedade: a possibilidade de o sistema atingir uma situação em que todas as células normais ficam permanentemente mortas.

As diferentes abordagens permitem combinar resultados obtidos por simulação com a capacidade de pesquisa simbólica do Z3.

Pesquisa de um estado de erro absorvente — `p2_contra_exemplo()`

A função `p2_contra_exemplo(N, MODO, kmax)` procura encontrar um estado de erro absorvente, ou seja, um estado em que todas as células normais estão mortas e nenhuma poderá voltar a ficar viva.

Para isso, define uma condição adicional (`sumidouro`), que verifica se a aplicação da regra de evolução a todas as células normais resulta sempre no valor morto.

O solver Z3 procura então um percurso que parta das condições iniciais e alcance um estado que satisfaça simultaneamente a condição de erro e a condição de sumidouro.

Se encontrar uma solução, extrai o percurso e valida-o através do motor concreto, confirmando também que todos os sucessores do estado final são iguais a esse mesmo estado.

Se não encontrar uma solução até ao limite definido, devolve `None` .

Esta função analisa especificamente a possibilidade de todas as células normais ficarem permanentemente mortas.

In [ ]:
```
def p2_contra_exemplo(N, MODO, kmax=12):
    """Leitura forte de P2: existe uma execução em que as células ficam
    permanentemente mortas.  Procura um estado de ERRO que seja um
```

```
SUMIDOURO
    (nenhuma célula normal muda a seguir, seja qual for a escolha).
    Num sumidouro, o 'lasso' é um auto-ciclo: qualquer escalonador é
justo."""

    def sumidouro(k):
        # com todas as normais mortas, o valor seguinte de cada célula é
regra_z3(...)
        # (só dá 1 se a borda lhe der exatamente 3 vizinhas vivas) ->
exigimos 0
        g = grelha_z3(N, k)
        return z3.And([z3.Not(regra_z3(g, i, j))
                       for i in range(1, N + 1) for j in range(1, N + 1)])

    s = z3.Solver()
    s.add(I_z3(N))
    for k in range(kmax + 1):
        s.push()
        s.add(erro_z3(N, k), sumidouro(k))
        if s.check() == z3.sat:
            traco = extrair(s.model(), N, k)
            s.pop()
            motor = GameOfLife_Engine(N, MODO)
            assert validatraco_z3(motor, traco)
# confirma no motor
            assert all(x == traco[-1] for x in motor.sucessores(traco[-1]))
# é sumidouro
            return k, traco
        s.pop()
        s.add(T_main(N, k, MODO))
    return None
```

Pesquisa de testemunhos através de simulação — `p2_testemunhas_sim()`

A função `p2_testemunhas_sim(N, modo, ...)` executa várias simulações do motor concreto, utilizando diferentes configurações da borda inicial.

Durante cada execução, regista para cada célula normal se já foi observada viva ou morta, guardando também a configuração da borda e o instante em que esse estado foi observado.

No modo síncrono, a grelha é atualizada simultaneamente em cada passo. No modo assíncrono, as células são atualizadas individualmente, percorrendo a grelha linha a linha.

No final, a função devolve três elementos:

- `viva` : dicionário com as células para as quais foi encontrado um testemunho vivo.

- `morta` : dicionário com as células para as quais foi encontrado um testemunho morto.
- `iniciais` : lista dos estados iniciais utilizados nas simulações.

Estes resultados permitem identificar quais as células que já possuem testemunhos e quais necessitam de pesquisa adicional através do Z3.

In [ ]:
```
def p2_testemunhas_sim(N, modo, bordas=(1, 0) + (None,) * 198, passos=150,
seed=0):
    """Leitura existencial de P2 por simulação: cada execução concreta é
uma prova
    de alcançabilidade.  Devolve viva, morta = {célula: (índice_da_borda,
passo)} e a lista
    dos estados iniciais usados (para reproduzir cada testemunho no
relatório)."""
    random.seed(seed)
    motor = GameOfLife_Engine(N, modo)
    celulas = [(i, j) for i in range(1, N + 1) for j in range(1, N + 1)]
    viva, morta, iniciais = {}, {}, []

    def regista(g, b, t):
        for (i, j) in celulas:
            (viva if g[i][j] else morta).setdefault((i, j), (b, t))

    for b, borda in enumerate(bordas):
        g = motor.estado_inicial(borda)
        iniciais.append([linha[:] for linha in g])
        regista(g, b, 0)
        if modo == "SYNC":
            for t in range(1, passos + 1):
                g = motor.sucessores(g)[0]
                regista(g, b, t)
        else:                                   # ASYNC: varrimento linha a
linha
            t = 0
            for _ in range(passos):
                for (i, j) in celulas:          # cada atribuição é UMA
transição
                    g[i][j] = motor.proximo_valor(g, i, j)
                    t += 1
                    (viva if g[i][j] else morta).setdefault((i, j), (b, t))
    return viva, morta, iniciais
```

Pesquisa simbólica de estados — `alcanca_viva()`

A função `alcanca_viva(N, MODO, cel, kmax, viva=True, timeout_ms=None)` procura determinar se uma célula específica consegue atingir um determinado estado.

Para isso, constrói um solver Z3 com as condições iniciais do sistema e verifica sucessivamente os diferentes instantes até ao limite `kmax` .

Dependendo do valor do parâmetro `viva`, procura um estado em que a célula esteja viva (`True`) ou morta (`False`).

Em cada instante, o solver verifica se existe um percurso que satisfaça a condição pretendida. Caso encontre uma solução, devolve o instante e o respetivo traço da execução.

Se não encontrar uma solução dentro do limite estabelecido, devolve `None`. Se o solver não conseguir determinar o resultado, devolve `UNKNOWN` e o respetivo motivo.

Esta função é utilizada para procurar os testemunhos que não foram encontrados durante as simulações do motor concreto.

In [ ]:
```python
def alcanca_viva(N, MODO, cel, kmax, viva=True, timeout_ms=None):
    """Procura um percurso em que a célula cel está viva (ou morta, se
viva=False)
    até kmax.  Usada só para as células que a simulação não cobriu."""
    i, j = cel
    solver = z3.Solver()
    if timeout_ms is not None:
        solver.set(timeout=int(timeout_ms))
    solver.add(I_z3(N))

    for k in range(kmax + 1):
        g = grelha_z3(N, k)[i][j]
        resultado = solver.check(g if viva else z3.Not(g))
        if resultado == z3.sat:
            return k, extrair(solver.model(), N, k)
        if resultado == z3.unknown:
            return "UNKNOWN", k, solver.reason_unknown()
        if k < kmax:
            solver.add(T_main(N, k, MODO))

    return None
```

Execução da verificação da Propriedade 2 — `p2_main()`

A função `p2_main(N, modo, kmax=20, timeout_ms=None)` coordena as diferentes abordagens utilizadas na verificação da segunda propriedade.

Inicialmente, executa `p2_contra_exemplo()` para procurar um estado de erro absorvente.

De seguida, executa `p2_testemunhas_sim()` para procurar, através de simulação, testemunhos vivos e mortos para cada célula normal.

Após esta etapa, identifica as células para as quais ainda não foi encontrado um dos estados pretendidos. Para essas células, utiliza `alcanca_viva()` para realizar uma pesquisa simbólica através do Z3.

Sempre que o solver encontra um percurso, este é validado através de `validatraco_z3()` antes de ser considerado um testemunho válido.

No final, são identificadas as células para as quais não foi possível encontrar testemunhos vivos ou mortos.

A propriedade é considerada verificada na interpretação existencial implementada quando todas as células normais possuem pelo menos um testemunho vivo e um testemunho morto.

Os resultados são organizados num dicionário, incluindo a pesquisa do estado absorvente, os testemunhos encontrados e a conclusão da verificação.

In [ ]:
```
def p2_main(N, modo,kmax=20, timeout_ms=None):
    """Executa a P2 nos dois modos: leitura forte e leitura existencial."""
    todos = [(i, j) for i in range(1, N + 1) for j in range(1, N + 1)]
    resultados = {}

    print(f"PROPRIEDADE 2 — MODO {modo}")

    print("\n[1] Leitura forte: existe uma execução com células
permanentemente mortas?")
    ce = p2_contra_exemplo(N, modo)
    if ce is None:
        print("  Nenhum contra-exemplo encontrado (inconclusivo).")
    else:
        k, _traco = ce
        print(f"  CONTRAEXEMPLO: estado de erro absorvente ao passo {k} "
              f"(traço validado no motor).")
        print("  Conclusão: P2 é FALSA nesta leitura.")


    print("\n[2] Leitura existencial: testemunho vivo e morto para cada
célula")
    passos = 150 if modo == "SYNC" else 40
    viva, morta, _iniciais = p2_testemunhas_sim(N, modo, passos=passos)
    print(f"  Simulação: vivas {len(viva)}/{N*N} | mortas
{len(morta)}/{N*N}")


    motor = GameOfLife_Engine(N, modo)
    for quer_viva, dic in ((True, viva), (False, morta)):
        for cel in [c for c in todos if c not in dic]:
            r = alcanca_viva(N, modo, cel, kmax, viva=quer_viva,
timeout_ms=timeout_ms)
            if r is not None and r[0] != "UNKNOWN":
                k, traco = r
                assert validatraco_z3(motor, traco)
                dic[cel] = (None, k)

    sem_viva = [c for c in todos if c not in viva]
    sem_morta = [c for c in todos if c not in morta]
    print("  Sem testemunho vivo:", sem_viva)
```

```
    print("  Sem testemunho morto:", sem_morta)

    completa = not sem_viva and not sem_morta
    if completa:
        print("  Conclusão: P2 VERIFICADA na interpretação existencial.")
    else:
        print("  Conclusão: INCONCLUSIVA para as células em falta.")

    resultados[modo] = {
        "contra_exemplo_forte": None if ce is None else ce[0],
        "viva": viva,
        "morta": morta,
        "sem_viva": sem_viva,
        "sem_morta": sem_morta,
        "verificada": completa,
    }

    return resultados
```

6. Execução e apresentação dos resultados no Marimo

A aplicação foi desenvolvida no Marimo, permitindo executar e visualizar interativamente o funcionamento do motor e a verificação das propriedades.

Através dos botões disponibilizados, o utilizador pode executar individualmente as diferentes figuras no motor ou iniciar a verificação das propriedades P1 e P2.

Quando é acionado o botão de verificação, são executadas as respetivas funções principais, `p1_main()` e `p2_main()`, responsáveis por coordenar os testes definidos para cada propriedade.

Os parâmetros de execução, como os limites de pesquisa e os tempos de espera do solver Z3, são definidos previamente no código.

Por fim, os resultados são apresentados diretamente na aplicação Marimo, permitindo acompanhar as conclusões obtidas durante a verificação do sistema.

In [ ]:
```
executar = mo.ui.run_button(label="Executar verificação (P1 e P2)")
executar
```

Executar verificação (P1 e P2)

In [ ]:
```
mo.stop(not executar.value)


# A P1 tem contraexemplos conhecidos até ao passo 9 nos dois modos.
KMAX_P1 = 100

# A P2 usa simulação para os testemunhos; KMAX_P2 e o timeout só se aplicam
```

```
# às células que a simulação não cobrir (consulta Z3 por célula).
KMAX_P2 = 20
TIMEOUT_P2_MS = 60000  # limite por consulta Z3; None desativa o limite

# Captura os prints das funções auxiliares e apresenta-os como output
Marimo.
# Assim, os resultados aparecem na própria aplicação, e não apenas no
terminal.
saida_p1 = StringIO()
with redirect_stdout(saida_p1):
    resultados_p1 = p1_main(N, MODO, kmax=KMAX_P1)

saida_p2 = StringIO()
with redirect_stdout(saida_p2):
    resultados_p2 = p2_main(N, MODO, kmax=KMAX_P2,
timeout_ms=TIMEOUT_P2_MS)
#LLM inicio
mo.vstack([
    mo.md("## Resultados da verificação"),
    mo.md("### Propriedade 1 (P1)"),
    mo.md("```text\n" + saida_p1.getvalue() + "\n```"),
    mo.md("### Propriedade 2 (P2)"),
    mo.md("```text\n" + saida_p2.getvalue() + "\n```"),
])
#LLM fim
```

Resultados da verificação

Propriedade 1 (P1)

```
PROPRIEDADE 1 — MODO ASYNC

[1] Verificação do candidato Inv = existe uma célula viva
Base: unsat
Passo indutivo: sat
  Base válida: todos os estados iniciais satisfazem Inv.
  Preservação falhou: existe uma transição Inv -> NOT Inv.
  Nota: esse estado pode não ser alcançável desde I.
  Candidato indutivo: NÃO

[2] BMC: procura de estado de erro até ao passo 100
  CONTRAEXEMPLO ENCONTRADO.
  Primeiro passo de erro encontrado: 9
  Células normais vivas por instante: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
  Traço validado no motor concreto: True
  Conclusão: a propriedade 1 é FALSA neste modo.
```

Propriedade 2 (P2)

```
PROPRIEDADE 2 — MODO ASYNC

[1] Leitura forte: existe uma execução com células permanentemente mor
tas?
  CONTRAEXEMPLO: estado de erro absorvente ao passo 9 (traço validado
no motor).
  Conclusão: P2 é FALSA nesta leitura.

[2] Leitura existencial: testemunho vivo e morto para cada célula
  Simulação: vivas 324/324 | mortas 324/324
  Sem testemunho vivo: []
  Sem testemunho morto: []
  Conclusão: P2 VERIFICADA na interpretação existencial.
```

Conclusões

A análise das propriedades foi realizada considerando os dois modos de execução do *Game of Life*: **síncrono (sync)** e **assíncrono (async)**. Os resultados obtidos permitem retirar as seguintes conclusões.

Modo Síncrono (Sync)

Propriedade 1 A Propriedade 1 revelou-se falsa neste modo de execução. Embora a verificação da condição inicial (*base*) tenha sido bem-sucedida, a verificação do passo indutivo falhou, uma vez que foi identificada uma transição de um estado que satisfaz o invariante (existência de pelo menos uma célula viva) para outro que não o satisfaz. Adicionalmente, a verificação por *Bounded Model Checking* (BMC), com uma profundidade máxima de 100 passos, encontrou um contraexemplo no passo 3. Neste percurso, o número de células vivas evoluiu segundo a sequência `[9, 3, 1, 0]`, demonstrando que é possível atingir um estado em que todas as células estão mortas. Este contraexemplo foi também validado pelo motor concreto de execução. **Propriedade 2** Relativamente à Propriedade 2, os resultados devem ser analisados de acordo com as duas interpretações consideradas:

- **Interpretação forte:** a propriedade é falsa, uma vez que foi encontrado um contraexemplo correspondente a um estado absorvente, atingido no passo 3, no qual existem células que permanecem permanentemente mortas.
- **Interpretação existencial:** através das várias execuções simuladas, foram encontrados testemunhos de que todas as 324 células podem assumir ambos os estados: vivo e morto. Ou seja, para cada célula, foi identificada pelo menos uma execução em que permaneceu viva e outra em que permaneceu morta.

Importa salientar que este resultado não constitui uma prova formal da Propriedade 2. Demonstra apenas que, nas execuções simuladas, foram encontrados testemunhos que evidenciam a capacidade de todas as células assumirem ambos os estados. Não garante, portanto, que este comportamento se verifique em todas as execuções possíveis do sistema.

Modo Assíncrono (Async)

Propriedade 1 Tal como no modo síncrono, a Propriedade 1 revelou-se falsa. Apesar de a condição inicial (*base*) ser válida, a preservação do invariante não foi demonstrada através do passo indutivo. A análise por BMC permitiu encontrar um contraexemplo no passo 9. Neste caso, o número de células vivas diminuiu progressivamente, seguindo a

sequência `[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]`, até atingir um estado sem células vivas. Este percurso foi igualmente validado pelo motor concreto de execução, confirmando a falsidade da propriedade neste modo. **Propriedade 2** À semelhança do modo síncrono, os resultados da Propriedade 2 variam consoante a interpretação adotada:

- **Interpretação forte:** a propriedade é falsa, tendo sido identificado um contraexemplo no passo 9, correspondente a um estado absorvente no qual existem células permanentemente mortas.
- **Interpretação existencial:** as várias execuções simuladas permitiram encontrar testemunhos de que todas as 324 células podem assumir ambos os estados: vivo e morto. Para cada célula, foi encontrada pelo menos uma execução em que permaneceu viva e outra em que permaneceu morta.

Tal como no modo síncrono, estes resultados constituem apenas evidência experimental obtida por simulação, não permitindo comprovar formalmente a Propriedade 2. Demonstram que foram encontrados exemplos de execuções em que todas as células assumem ambos os estados, mas não garantem que tal aconteça em todas as execuções possíveis.

Síntese Final

Os resultados obtidos demonstram que os modos síncrono e assíncrono apresentam comportamentos distintos quanto à evolução temporal do sistema, nomeadamente no número de passos necessários para atingir um estado sem células vivas. Em ambos os modos, a Propriedade 1 é falsa, uma vez que foram encontradas execuções que conduzem a um estado em que todas as células estão mortas. No modo síncrono, esse estado é atingido ao passo 3, enquanto no modo assíncrono é atingido ao passo 9. Relativamente à Propriedade 2, a análise evidencia a importância de distinguir as duas interpretações consideradas. Na interpretação forte, a propriedade é falsa em ambos os modos, devido à existência de contraexemplos que conduzem a estados absorventes. Por outro lado, na interpretação existencial, as simulações permitiram encontrar testemunhos de que todas as 324 células podem assumir ambos os estados, vivo e morto. Contudo, estes testemunhos representam apenas evidência experimental, não constituindo uma demonstração formal da propriedade nem garantindo o comportamento observado para todas as execuções possíveis.

Testes com Pytest

Na pasta `tests` encontram-se quatro ficheiros dedicados aos testes do programa:

- `Auxiliares`: contém funções de apoio utilizadas pelos diferentes testes.
- Os restantes três ficheiros contêm os testes específicos de cada componente do programa, correspondendo às funcionalidades identificadas nos respetivos

nomes.

Ficheiros:

- Link para auxiliares
- Testes Modelo z3
- Testes Motor
- Testes Propriedades

Como executar os testes

Para executar todos os testes, deve abrir-se um terminal na pasta principal do projeto (`TP2.1`), com o ambiente Conda ativado, e executar o seguinte comando:

```
uv run pytest -v
```

Este comando executa automaticamente todos os testes encontrados, apresentando os resultados de cada um e indicando se foram concluídos com sucesso ou se ocorreram falhas.

Uso de LLM

Ao longo do desenvolvimento deste trabalho, foram utilizadas ferramentas de Inteligência Artificial (IA), nomeadamente o **Claude** e o **ChatGPT**, como ferramentas de apoio ao esclarecimento de conceitos, à definição de estratégias, à estruturação do código e à sua implementação.

Importa salientar que as secções relativas à animação do *Game of Life* e aos testes com `pytest` foram integralmente geradas com recurso a IA. Adicionalmente, as estratégias utilizadas para a demonstração das Propriedades 1 e 2 foram sugeridas e posteriormente revistas e validadas com o auxílio destas ferramentas.

- Claude 1
- Claude 2
- Claude 3
- ChatGPT 1

O grupo assume integralmente a responsabilidade por todo o trabalho desenvolvido, incluindo a revisão, validação e integração dos conteúdos e do código produzidos com o auxílio de ferramentas de Inteligência Artificial.

In [ ]: